
FULL-AGENT KERNEL GENERATION FOR FLASHINFER @ MLSys 2026

Chenyu Ma¹ Yue Shui¹ Hangfei Xu¹ Shengzhao Wen^{† 1} Yanpeng Wang^{† 1}

ABSTRACT

This report describes our Full-Agent submissions to the FlashInfer AI Kernel Generation Contest at MLSys 2026. We modify Baidu Baige LoongFlow into an autonomous CUDA/Triton kernel-search system and evaluate it on five tasks from the GDN, DSA, and MoE tracks. The system uses a directed Plan-Execute-Summarize loop: it proposes kernel changes, writes candidate code, invokes the official evaluator, summarizes failures and successes, and updates an evolutionary database without human intervention after task setup. All five final kernels pass correctness, with submitted kernel latencies of 0.0084 ms for GDN decode, 0.7020 ms for GDN prefill, 0.0272 ms for DSA sparse attention, 0.0356 ms for DSA top-k indexing, and 1.448 ms for MoE FP8 routing. The archived traces provide both final solutions and evidence of how autonomous search converged.

1 INTRODUCTION

Efficient LLM serving is increasingly determined by a small set of GPU kernels rather than by the model graph alone. Operators such as routed MoE GEMMs, sparse attention over paged KV caches, top-k index construction, and recurrent state updates expose irregular memory access, mixed precision, short decode regimes, and long-context regimes. The FlashInfer AI Kernel Generation Contest turns these pressures into a controlled systems benchmark: each submitted kernel must compile in the target environment, preserve the reference semantics, and reduce measured latency over the official workload distribution.

LLM-based kernel generation is therefore not a one-shot synthesis problem. A plausible CUDA or Triton candidate can fail because of packaging drift, unsupported intrinsics, numerical mismatch, shape-specific regressions, or a local optimization that does not survive the full evaluator. The agent-assisted report for the same contest frames this as a harness-engineering problem: humans design the evaluation and promotion loop, while code agents propose bounded candidate patches. This report studies the complementary question: how far can the loop be closed by the agent itself after task setup?

The two reports therefore make different claims. The agent-assisted paper evaluates human-curated retained kernels under a harness/controller workflow, while this report isolates the autonomous Full-Agent pathway. We use the same compile, correctness, and latency discipline and add the sup-

plied FlashInfer baseline to trajectory plots for context, but the primary object here is trace-level autonomy and auditability.

We build a Full-Agent system by modifying Baidu Baige LoongFlow into an autonomous FlashInfer kernel-search framework. The system keeps the Plan-Execute-Summarize structure of LoongFlow, replaces the task layer with FlashInfer CUDA/Triton contracts, and records every plan, generated candidate, evaluator result, summary, and selected parent in an evolutionary trace database. After launching a task, no human edits intermediate kernels, selects parents, filters failures, or steers the next implementation direction.

Our approach includes the following design contributions.

LoongFlow-Derived Autonomous PES Runtime. We adapt LoongFlow’s directed evolutionary framework into a kernel-search runtime with planner, executor, evaluator, summarizer, database, and finalizer roles. The same runtime structure is reused across GDN, DSA, and MoE, while the task prompts, evaluator bindings, and final export paths vary by contest definition.

Evidence-Conditioned Prompt and Memory Policy. The planner prompt is rebuilt at every iteration from the fixed operator contract plus evolving evidence: parent solutions, archived evaluator records, compiler failures, correctness failures, latency measurements, and summarizer lessons. The prompt is therefore a state-dependent search policy rather than a static instruction.

Shape-Specialized Kernel Search Without Manual Intervention. The agent is allowed to introduce dispatch paths or specialized kernels for distinct shape regimes. This mirrors the agent-assisted observation that full-distribution

[†]Equal supervision. ¹Baidu Inc. Correspondence to: <machenyu01@baidu.com> <shuiyue@baidu.com>.

performance often requires shape-aware routing, but here the routes are generated and validated inside the autonomous loop rather than inserted by a human.

Block-Level Multi-Model Switching. Because the search state lives in the trace database rather than in a single chat context, the system can switch planner or executor models at block boundaries. We use this to support broad exploration, code-heavy refinement, and conservative review phases while preserving parent ids, evaluator logs, and handoff summaries.

Auditable Full-Agent Trajectories. The output is not only a final kernel. Each repository contains iteration-level planner, executor, evaluator, and summarizer artifacts. These traces make the autonomous search inspectable: reviewers can see which candidate failed, which parent was preserved, and where the agent shifted from broad exploration to narrow exploitation.

The report covers five definitions across the three contest tracks. GDN decode and prefill are recurrent state-update kernels; DSA sparse attention and top-k indexer stress sparse memory and selection; MoE FP8 routing combines exact routing semantics with grouped FP8 matrix multiplication. All final submitted kernels pass correctness, and Section 3 reports kernel latency as the primary metric.

2 FULL-AGENT SYSTEM DESIGN AND WORKFLOW

2.1 LoongFlow-Derived Stack

Our system is derived from Baidu Baige’s open-source [LoongFlow](#) framework. The modification is architectural rather than cosmetic: we keep LoongFlow’s directed evolutionary substrate and replace its generic task interface with FlashInfer operator contracts, CUDA/Triton executors, official-style evaluator bindings, trace packaging, and final solution export.

Figure 1 should be read as a stack, not as an iteration trace. The top layer contains task-specific launchers and prompts for GDN, DSA, and MoE. The middle layer provides the PES roles inherited from LoongFlow: planning, execution, evaluation, summarization, storage, and finalization. The bottom layer provides reusable model, memory, tool, message, and observability components. This separation is what makes the system reusable across all five kernels while keeping each task’s CUDA/Triton contract isolated.

2.2 Evidence Interface: Prompt, Evaluator, and Trace Memory

Each task starts from a fixed contract: operator semantics, tensor shapes, dtypes, layout constraints, entry-point signature, numerical tolerances, and allowed implementation language. GDN and DSA use CUDA solution modules, while MoE uses a Triton implementation wrapped by the contest adapter.

At every iteration the planner prompt is reconstructed from the task contract and trace memory. Early prompts are architecture-seeking and ask for broad implementation strategies. Later prompts become parent-relative: preserve a proven split-K path, rebase onto a correct grouped-GEMM parent, add one isolated autotuning change, or avoid a failure pattern observed in the last evaluator block. The prompt is therefore a state-dependent policy, not a static instruction.

The evaluator is the sole promotion gate. A candidate is useful only if it compiles, passes correctness, produces a valid latency measurement, and can be archived with its generated source and metadata. Failed candidates are not discarded as noise; they become negative evidence that shapes later prompts. Profiler evidence, when archived, is folded into the same summarizer channel as an advisory signal rather than as a separate control path.

2.3 Autonomous Search Policy

The autonomous policy is the part that closes the loop after launch. The planner selects a parent or frontier candidate from the database and proposes one bounded implementation direction. The executor materializes the change in CUDA or Triton. The evaluator measures compile, correctness, and latency. The summarizer converts the result into reusable constraints, and the database updates the next frontier.

This policy differs from the agent-assisted workflow in one central way: the human does not perform plateau recovery during the run. The agent must decide from trace evidence whether to preserve a known-good parent, retry a failed route with smaller scope, or introduce a new shape-specialized path. Manual effort is limited to task setup, API configuration, and launching the repository-specific script.

Shape specialization is allowed inside this policy because the contest workloads are not governed by a single bottleneck. GDN decode benefits from fixed-dimension CUDA specialization and dispatch over known head/value layouts. GDN prefill benefits from register-resident recurrent state. DSA sparse attention keeps both a fused online-softmax path and a split-K path. DSA top-k uses separate strategies for short, medium, and long sequences. MoE preserves exact routing semantics while specializing the two grouped

Modified LoongFlow Full-Agent Stack

Autonomous kernel optimization through iterative plan–execute–evaluate–summarize–store loop

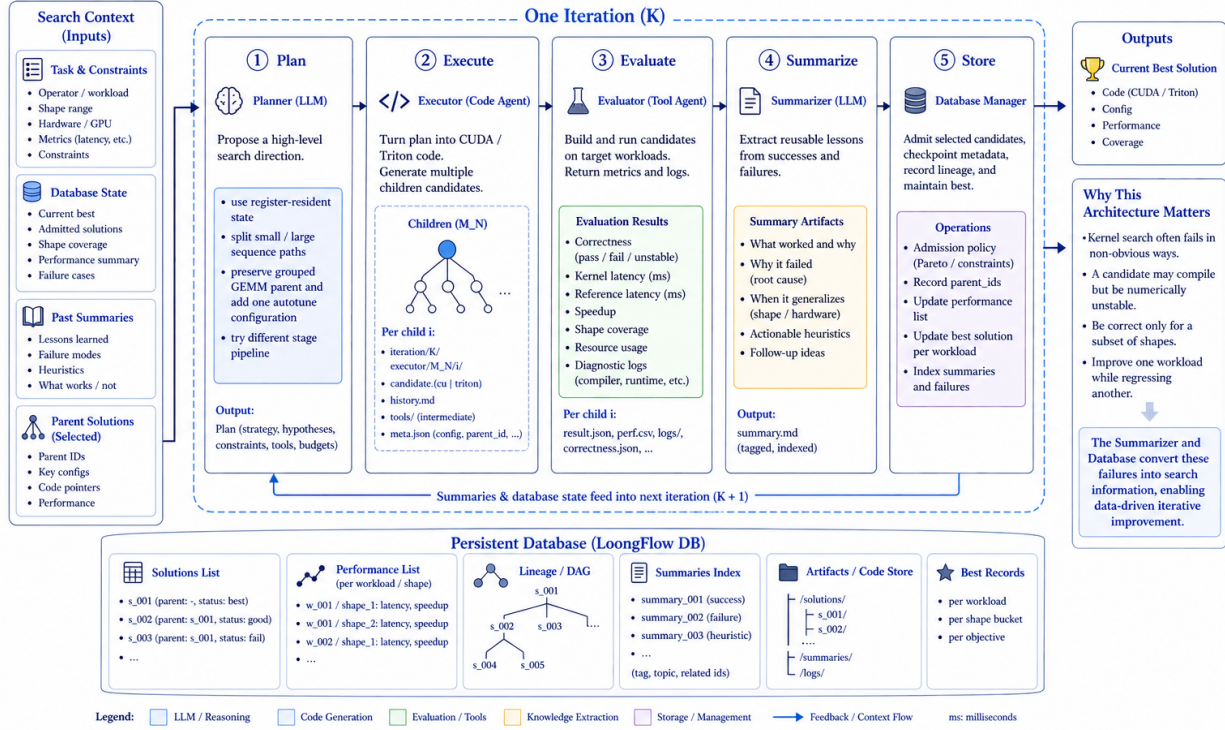


Figure 1. Modified Baidu LoongFlow full-agent stack. The benchmark-specific application layer changes across GDN, DSA, and MoE, while the PES framework layer and shared components are reused across all five autonomous kernel searches.

FP8 GEMMs. These branches are produced inside the autonomous loop and validated by the evaluator.

2.4 Block-Level Model Switching

The search state is stored outside any single LLM call, so the same PES run can continue after switching the planner or executor model. This enables stage-wise policies such as Gemini-Pro-3.1 for broad exploration, GPT-5.4 for code-heavy refinement, and Claude-3.6-Opus-style models for conservative review. Switching is performed only at block boundaries, for example after a fixed 10- or 20-iteration block or after the database contains a valid parent, so the system does not mix model assumptions inside one evaluator block.

Figure 2 summarizes the control rule. Two iterations form one search block under a fixed model. The system compares the block-start champion with the block-end best valid candidate, then either keeps the model, switches models with a handoff note, or stops when repeated plateaus make the best solution stable. Measurement feedback is folded back into the next planner prompt, while compile success, correctness, and latency remain the promotion

gate.

The auxiliary experiments in the model-switching directory study this policy independently from the final contest trajectories. In the single-model 20-round run, the retained speedup plateaus near 11.7x. In the multi-model experiment on DSA sparse attention, block-level promote/reject/diagnose decisions continue the search to about 27.54x by round 25. We therefore treat multi-model switching as a control-policy enhancement: it changes the model backend, but leaves database checkpoints, parent ids, summaries, evaluator logs, and task contracts intact. The corresponding plots are included in Appendix B because they support the agent-control design rather than the official final kernel-latency table.

Block Search and Model Handoff

Two iterations form one block; switching is decided only after the block is evaluated.

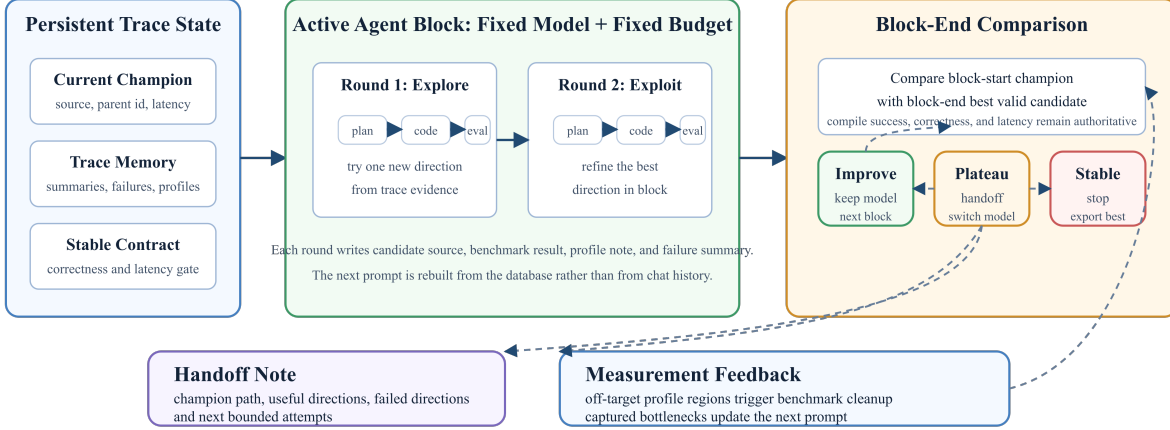


Figure 2. Block-level handoff loop. Model changes happen only at block boundaries, and trace evidence updates the next prompt or handoff note.

2.5 Reproducibility and Trace Layout

Each repository contains both final submission files and complete full-agent traces. The top-level trace mirrors are provided so reviewers do not need to descend into the agent output directory. The reproduction flow is intentionally uniform across all tasks:

```
# Select track: <track> = gdn, dsa, or moe.
cd flashinfer-bench-loongflow-fullagent-<track>

# Create and activate the Python environment.
uv venv .venv --python 3.12
source .venv/bin/activate
uv pip install -e .

# Configure the LLM endpoint.
export LLM_API_KEY="..."
export LLM_BASE_URL="..."

# Run one task-specific launcher in the selected
# repository.
./run_<task>.sh
```

Each launcher renders environment-variable-driven configuration, creates or uses a task workspace, and calls the Math-Agent PES entry point with the task prompt, initial JSON task definition, evaluator file, and task config. Once launched, the same PES loop handles planning, code generation, evaluation, summarization, checkpointing, and parent selection without manual intervention. The complete trace schema and winning checkpoint paths are listed in Appendix C.

3 EXPERIMENTAL EVALUATION

Table 1 summarizes the final submitted kernels with kernel latency as the primary metric. All final kernels pass

correctness. The table reports the submitted checkpoint for each task; Section 5 separately visualizes the executor’s best evaluated candidate at each archived iteration.

The results cover three qualitatively different optimization regimes. GDN decode is a small fixed-shape recurrent update; its final latency of 0.0084 ms reflects aggressive template specialization, register-level reductions, and dispatch over known head/value layouts. GDN prefill is memory-traffic dominated; the final 0.7020 ms kernel is obtained by keeping state rows register-resident across the token loop instead of repeatedly loading and storing the recurrent state tensor. These two GDN results indicate that the agent did not apply one generic CUDA recipe, but adapted to the decode/prefill workload split.

The two DSA tasks stress different aspects of sparse attention. Sparse attention reaches 0.0272 ms in the submitted checkpoint by combining a fused online-softmax path with a robust split-K fallback; this is important because several faster-looking candidates were shape fragile. The top-k indexer reaches 0.0356 ms with a length-aware score/sort decomposition: direct enumeration for short inputs, block-level sorting for medium inputs, and segmented device sorting for long inputs. MoE is the densest workload and the least reducible to a single fused kernel; its 1.448 ms final latency comes from preserving exact routing semantics while moving the routed GEMMs onto FP8 tensor-core-friendly Triton paths. The appendix reports the corresponding per-iteration reference latency and speedup values, since those baselines vary across archived benchmark runs.

The submitted source files follow the contest layout: GDN and DSA use task-local CUDA solution directories, while

Table 1. Final submitted kernel latency by task. Lower kernel latency is better; reference latency is included only as contextual baseline.

Track	Task	Impl.	Kernel latency	Reference latency	Best iter.	Solution ID
GDN	Decode	CUDA	0.0084 ms	116.783 ms	10	01bea42b
GDN	Prefill	CUDA	0.7020 ms	1236.742 ms	10	ed639eed
DSA	Sparse attention	CUDA	0.0272 ms	3.421 ms	20	94736692
DSA	Top-k indexer	CUDA	0.0356 ms	3.256 ms	14	dc51d5fc
MoE	FP8 routing	Triton	1.448 ms	16.542 ms	40	6da80c61

MoE uses a task-local Triton solution directory. Exact task paths are listed in Appendix C.

4 FINAL KERNEL ANALYSIS

4.1 GDN Decode

GDN decode performs a recurrent single-token state update used in linear attention models. The state layout is k-last over B, HV, V, and K, and the operation expands query/key heads for GVA mode. The final CUDA design specializes hot shapes with templated kernels while keeping a generic fallback path. The fast kernel maps blocks over batch, value head, and value-dimension tiles. Within a block, lanes cooperatively load BF16 query/key data, compute dot products with warp-level reductions, update FP32 state, and write BF16 output.

The most important design choice is to exploit the fixed small dimensions. Instead of treating the operator as a generic matrix computation, the kernel uses compile-time specialization for the K dimension, vectorized loads for BF16 and FP32 state, and deterministic dispatch among template instantiations. Gate terms g and β are computed once per block and reused. Warp shuffle primitives reduce q-by-k, q-by-state, and k-by-state products without shared-memory reductions.

The search trajectory shows two phases. Iterations 1-4 explored increasingly parallel V-dimension mappings. Iteration 5 introduced a much stronger fast path, sharply reducing kernel latency. Iterations 6-10 refined runtime dispatch and template coverage, ending at 0.0084 ms final kernel latency.

4.2 GDN Prefill

GDN prefill processes multi-token chunks and updates recurrent state across each sequence. The main bottleneck is global-memory traffic for the state tensor with shape `num_seqs` by 8 by 128 by 128. The final kernel addresses this by keeping assigned state rows resident in registers for the full token loop. Query, key, and value chunks are staged through shared memory, and final state is written once at sequence end.

The kernel uses a shared-memory transpose to initialize

register-resident state efficiently, vectorized BF16 unpacking, four-float vector register math, fast math intrinsics for gate computation, and a carefully scheduled FMA loop for state update and output accumulation. This design matches the task prompt’s key optimization insight: prefill benefits more from avoiding state reload/store traffic than from treating every token independently.

The trajectory improved almost monotonically. The first iteration established the register-resident baseline at 6.3228 ms. Iterations 2-7 added double buffering, vectorized register math, and warp-shuffle broadcast. Iterations 8-10 added asynchronous or vectorized memory pipelining, reaching 0.7020 ms final kernel latency.

4.3 DSA Sparse Attention

DSA sparse attention computes attention over top-k sparse page indices. The query has content and positional parts, and the key/value cache is paged. The final CUDA solution keeps a two-path architecture: a fused online-softmax path for favorable workloads, and a split-K path with exact merge reduction for smaller or partitioned workloads.

In the fused path, each block handles a token and head group. Sparse indices are loaded in tiles, valid pages are staged into shared memory, BF16 content and positional dot products are accumulated in FP32, and online softmax maintains m and l without materializing all logits. The output is accumulated in registers and converted to BF16 at the end. Invalid sparse tiles are skipped early, reducing unnecessary shared-memory loads.

The split-K path partitions the sparse index range, computes partial online-softmax states, and merges them exactly. This was important for robustness. Several candidate designs improved one regime but regressed another; the final solution preserved the known-good large path while improving the small-path layout and merge behavior. The best checkpoint at iteration 20 achieved 0.0272 ms final kernel latency.

4.4 DSA Top-K Indexer

The DSA top-k indexer computes scores between FP8 query vectors and a paged FP8 key-index cache, applies ReLU and weighted head reduction, and returns the top-

2048 global token indices. The final solution separates scoring from selection and uses sequence-length-aware dispatch.

For short sequences, when L is no larger than top- k , the kernel simply enumerates valid token indices and pads with negative-one sentinels. For medium sequences, it computes scores and uses CUB block radix sorting within one block per batch item. For long sequences, it uses CUB segmented device radix sorting and a follow-up extraction kernel. The scoring kernel respects the DeepGEMM FP8 page format: key data are stored as FP8 bytes with per-token float scales, so the kernel explicitly dequantizes each token before scoring. Query data and weights are staged in shared memory, and warp reductions combine head contributions.

The early iterations attempted more aggressive CUB and fused-selection designs and failed to compile or underperformed. The breakthrough came when the agent used structural dispatch: direct enumeration for short inputs, block-level sorting for medium inputs, and device-level segmented sorting for long inputs. This tiered design reached 0.0356 ms final kernel latency at iteration 14.

4.5 MoE FP8 Block-Scale Routing

The MoE task is a routed FP8 block-scale computation implemented in Triton. The final solution preserves exact PyTorch routing semantics: sigmoid scores with bias, group selection, top- k expert selection, routing-weight normalization, local expert filtering, sorting by local expert, and accumulation back to token order.

After routing, the implementation launches two grouped Triton GEMMs. GEMM1 multiplies FP8 hidden states by expert weights with block scales and writes a Tk by $2I$ intermediate. GEMM2 applies SwiGLU, dynamically quantizes each activation tile to FP8, multiplies by FP8 expert weights, applies routing weights, and writes a per-routed-token output. The final output is produced with PyTorch index-add accumulation, and the submitted file includes a destination-passing-style adapter around the agent-returned implementation.

The MoE trajectory is the longest and least monotonic. Early iterations found correct grouped GEMM baselines around 3 ms. Middle iterations explored correctness-first fallback paths and exact PyTorch oracle components; these were slower but clarified routing and numerical constraints. The decisive improvements came from FP8 Tensor Core GEMM2, Triton loop pipelining, expanded autotuning, and conservative rebases onto proven parents. Iteration 40 reached 1.448 ms final kernel latency.

5 SEARCH TRAJECTORY ANALYSIS

The following five figures are generated from the archived iteration-level evaluator records and plot running-best valid kernel latency. The y-axis is log-scale latency in milliseconds; downward movement indicates improvement. Failed or non-improving iterations carry forward the previous best without failure markers, and DSA top- k begins at its first successful evaluation. The dashed gray line is the supplied FlashInfer baseline mean latency used for context in the common-baseline comparison; the plotted series remains the Full-Agent trace latency. Open circles mark the largest latency drops, and the red star marks the lowest observed latency. Failed candidates remain in the trace and appendix tables even when they are omitted from the plotted curves.

For **GDN decode**, the decisive drop occurs at iteration 5, where the agent introduces a shape-specialized fast path and reaches the 0.0084 ms regime. Later iterations mostly preserve this design while refining template coverage and dispatch.

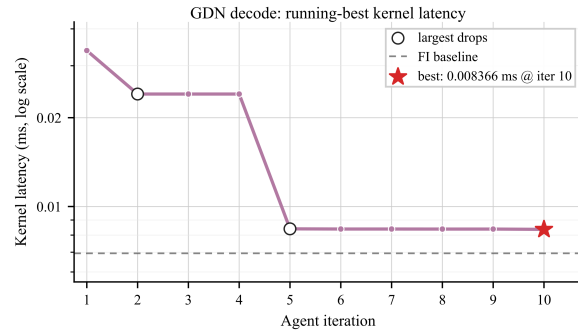


Figure 3. GDN decode running-best latency from archived evaluator records. The dashed line marks the supplied FlashInfer baseline.

For **GDN prefill**, the search is nearly monotonic after the register-resident-state idea appears. Double buffering, vectorized register math, and memory pipelining move latency from 6.3228 ms to a best observed 0.6889 ms at iteration 9; the submitted iteration-10 kernel remains close at 0.7020 ms.

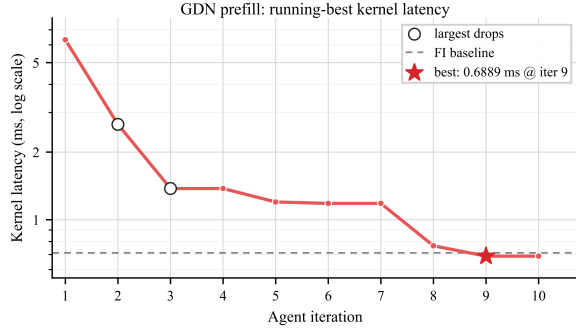


Figure 4. GDN prefill running-best latency from archived evaluator records. The dashed line marks the supplied FlashInfer baseline.

For **DSA sparse attention**, the search first moves from FlashAttention-like tiling to split-K/asynchronous dispatch, then makes a second transition at iteration 11 to a fused online-softmax path. The submitted checkpoint at iteration 20 keeps the robust two-path design at 0.0272 ms, while the trace also records later low-latency sibling evidence.

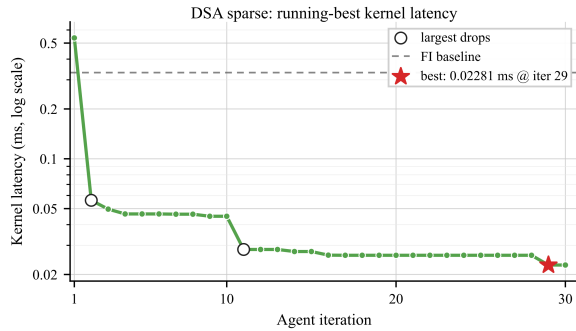


Figure 5. DSA sparse attention running-best latency from archived evaluator records. The dashed line marks the supplied FlashInfer baseline.

For **DSA top-k indexer**, the first successful point is iteration 4 after three failed attempts. The major improvement comes from length-aware dispatch: a short-sequence bypass and selective CUB sorting at iteration 12, followed by the submitted three-tier design at iteration 14.

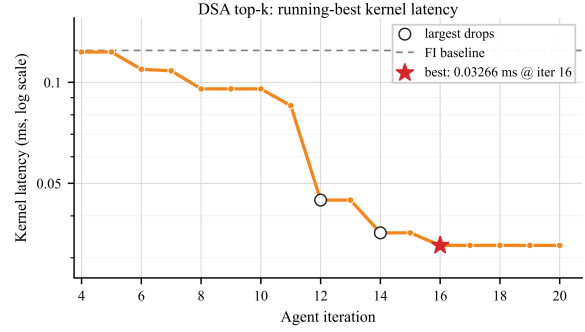


Figure 6. DSA top-k indexer running-best latency from archived evaluator records. The x-axis begins at the first successful evaluation, iteration 4; the dashed line marks the supplied FlashInfer baseline.

For **MoE**, early valid grouped-GEMM candidates are already around 3 ms, so improvement is less abrupt than in GDN or DSA. The decisive phase is iterations 26-32, where FP8 tensor-core GEMM2, Triton loop pipelining, and expanded autotuning reduce latency into the 1.5 ms range before the final 1.448 ms checkpoint.

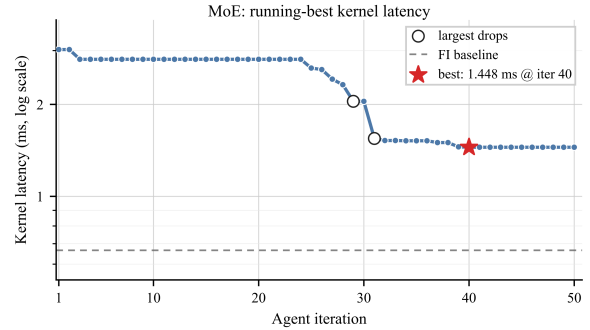


Figure 7. MoE running-best latency from archived evaluator records. The dashed line marks the supplied FlashInfer baseline.

6 CONCLUSION

The Full-Agent system demonstrates that a LoongFlow-derived PES runtime can produce correct FlashInfer kernels across GDN, DSA, and MoE without manual intervention after task setup. Across five tasks, the same planner, executor, evaluator, summarizer, and database loop preserves evidence, rebases on successful parents, and advances kernel search through measured compile, correctness, and latency feedback.

APPENDIX

Appendix tables and supporting plots are separated from the main report and are not counted toward the main-page limit.

A PER-TASK ITERATION TABLES

Appendix tables are separated from the main report and are not counted toward the main-page limit.

The following per-task tables are parsed from the archived iteration-level evaluator records. Numeric rows report the executor-best valid kernel latency, the corresponding reference latency, and speedup for that iteration. Status rows indicate that no valid kernel latency was produced in that iteration.

A.1 GDN Decode

Table 2. Per-iteration executor-best evaluation for GDN Decode. Numeric rows report valid latency measurements; status rows indicate that no valid kernel latency was produced.

Iteration	Kernel latency (ms)	Reference latency (ms)	Speedup
1	0.033777	24.7522	705.54x
2	0.024055	25.4192	970.05x
3	0.0254472	25.1327	882.89x
4	0.0345251	23.9155	547.62x
5	0.00840131	107.65	7770.14x
6	0.00838952	115.79	8272.10x
7	0.00839194	111.74	7960.39x
8	0.00838655	114.566	8149.40x
9	0.00840779	112.733	8114.77x
10	0.00836628	116.783	8379.51x

A.2 GDN Prefill

Table 3. Per-iteration executor-best evaluation for GDN Prefill. Numeric rows report valid latency measurements; status rows indicate that no valid kernel latency was produced.

Iteration	Kernel latency (ms)	Reference latency (ms)	Speedup
1	6.32278	1382.85	106.47x
2	2.6561	1578.33	285.21x
3	1.37723	1194.45	430.11x
4	1.39963	1227.12	445.09x
5	1.19979	1206.85	500.95x
6	1.18238	1211.15	516.24x
7	1.18509	1280.7	547.69x
8	0.76595	1193.13	777.88x
9	0.688875	1229.59	886.55x
10	0.701963	1236.74	888.46x

A.3 DSA Sparse Attention

Table 4: Per-iteration executor-best evaluation for DSA Sparse Attention. Numeric rows report valid latency measurements; status rows indicate that no valid kernel latency was produced.

Iteration	Kernel latency (ms)	Reference latency (ms)	Speedup
1	0.537143	2.1731	6.34x
2	0.0561113	2.23759	39.44x
3	0.049647	2.10315	41.68x

Continued on next page

Iteration	Kernel latency (ms)	Reference latency (ms)	Speedup
4	0.0464298	2.03053	42.98x
5	0.0470891	2.14625	44.72x
6	0.0487129	2.17633	43.70x
7	0.046268	2.11981	45.07x
8	0.0472993	2.12515	44.13x
9	0.0449555	1.95885	42.74x
10	Exec. fail	–	–
11	0.0283224	2.90363	104.00x
12	0.540023	3.18509	8.46x
13	0.769631	2.88654	6.77x
14	0.0275263	2.66897	96.42x
15	0.0276897	3.18241	114.49x
16	0.0261725	3.03278	115.00x
17	0.0261247	3.02655	116.91x
18	0.847041	2.84386	6.70x
19	0.0833066	2.93608	42.60x
20	0.0271814	3.4207	126.35x
21	0.0281254	2.19067	78.82x
22	Exec. fail	–	–
23	0.0268066	2.43738	90.75x
24	Exec. fail	–	–
25	0.0270459	2.49204	92.06x
26	0.0270235	2.73689	100.88x
27	0.0267601	2.58914	96.82x
28	0.0262193	2.59623	99.81x
29	0.022811	2.66034	114.81x
30	Missing	–	–

A.4 DSA Top-K Indexer

Table 5: Per-iteration executor-best evaluation for DSA Top-K Indexer. Numeric rows report valid latency measurements; status rows indicate that no valid kernel latency was produced.

Iteration	Kernel latency (ms)	Reference latency (ms)	Speedup
1	Exec. fail	–	–
2	Exec. fail	–	–
3	Exec. fail	–	–
4	0.123126	4.03755	34.51x
5	0.130025	4.29448	34.05x
6	0.109342	4.05491	37.53x
7	0.108242	4.05638	41.06x
8	0.0955908	3.81934	39.43x
9	0.0969991	3.9724	42.36x
10	0.10181	4.02982	39.81x
11	0.0852032	3.36701	40.17x
12	0.0445653	2.78833	89.70x
13	0.0455991	2.90609	81.57x
14	0.0355926	3.25596	95.64x
15	0.0365658	3.24539	93.64x
16	0.0326586	2.53297	86.01x
17	0.0570398	2.80207	78.60x
18	0.0443092	2.4941	81.12x
19	0.0336719	2.85315	90.04x
20	0.0472929	2.85856	79.74x

A.5 MoE FP8 Routing

Table 6: Per-iteration executor-best evaluation for MoE FP8 Routing. Numeric rows report valid latency measurements; status rows indicate that no valid kernel latency was produced.

Iteration	Kernel latency (ms)	Reference latency (ms)	Speedup
1	3.02372	18.6143	8.35x
2	Exec. fail	–	–
3	2.80971	18.3761	8.63x
4	3.02236	18.0305	8.13x
5	3.35866	18.4986	7.65x
6	Exec. fail	–	–
7	3.37411	18.4403	7.37x
8	3.3571	18.3852	7.63x
9	Valid. fail	–	–
10	Valid. fail	–	–
11	14.5428	18.4029	1.52x
12	2.85411	18.7479	8.03x
13	14.0298	18.4892	1.64x
14	8.28335	18.7339	2.39x
15	61.6742	18.4297	0.86x
16	Valid. fail	–	–
17	7.5761	17.9724	2.77x
18	18.3342	18.3916	1.12x
19	11.5477	19.052	1.84x
20	14.94	18.5387	1.36x
21	7.82537	17.53	2.79x
22	14.3034	18.2693	1.58x
23	7.98493	18.2293	2.42x
24	285.524	18.6298	0.06x
25	2.63012	18.2359	8.72x
26	2.60023	18.6474	10.25x
27	2.4195	18.9071	10.60x
28	2.32025	17.4429	12.01x
29	2.04679	17.0988	11.66x
30	2.16917	17.8044	12.23x
31	1.54685	17.0908	15.34x
32	1.52336	16.7998	15.40x
33	1.62434	17.251	14.33x
34	1.52043	16.8896	15.52x
35	Valid. fail	–	–
36	1.64844	17.3404	14.39x
37	1.49991	16.8488	15.48x
38	1.61179	17.5509	14.41x
39	1.45231	16.8622	15.90x
40	1.44809	16.5418	15.98x
41	1.60015	17.155	14.87x
42	1.53904	17.0664	15.29x
43	12.6417	16.6379	1.87x
44	59.2486	17.1109	1.16x
45	1.47252	16.6591	15.76x
46	1.49139	16.6587	15.59x
47	1.63865	17.4669	14.35x
48	1.58952	16.7324	14.55x
49	1.55833	16.8981	14.80x
50	1.47221	16.5215	15.76x

B MULTI-MODEL SWITCHING EXPERIMENT

The following auxiliary plots come from the model-switching experiment directory. They are not used as final contest results; they support the agent-control design in Section 2.4. The single-model run shows a 20-round plateau near 11.7x, while the multi-model run continues the DSA sparse-attention search with block-level promote, reject, and diagnose decisions.

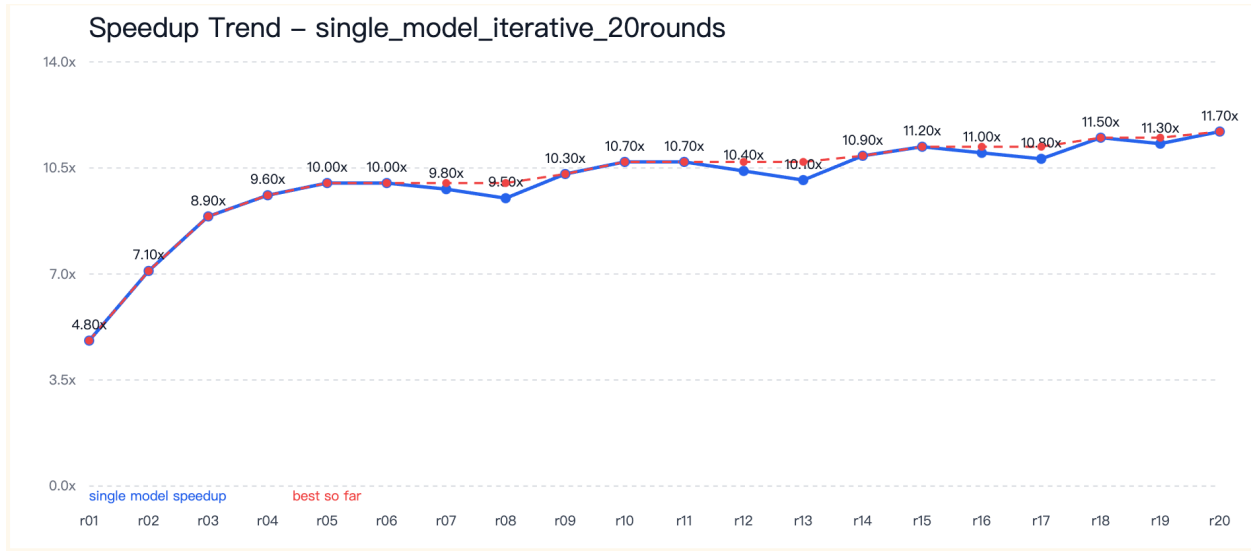


Figure 8. Auxiliary single-model 20-round speedup trajectory.

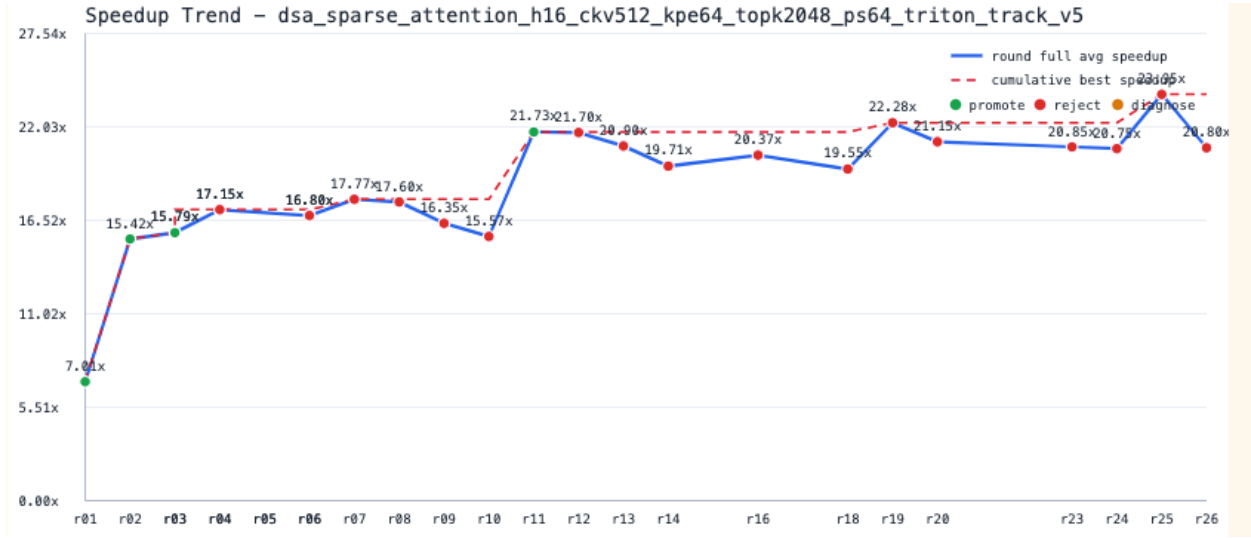


Figure 9. Auxiliary multi-model trajectory for DSA sparse attention.

C TRACE SCHEMA AND WINNING CHECKPOINTS

The final submission source files are:

Task	Submitted solution directory
GDN decode	flashinfer-bench-loongflow-fullagent-gdn/gdn_decode_qk4_v8_d128_k_last/solution/cuda
GDN prefill	flashinfer-bench-loongflow-fullagent-gdn/gdn_prefill_qk4_v8_d128_k_last/solution/cuda
DSA sparse attention	flashinfer-bench-loongflow-fullagent-dsa/dsa_sparse_attention_h16_ckv512_kpe64_topk2048_ps64/solution/cuda
DSA top-k indexer	flashinfer-bench-loongflow-fullagent-dsa/dsa_topk_indexer_fp8_h64_d128_topk2048_ps64/solution/cuda
MoE	flashinfer-bench-loongflow-fullagent-moe/solution/triton

The trace schema is consistent across tasks:

- `database/checkpoints/checkpoint-checkpoint-iter-{K}-{N}/best_solution.json` stores the best solution at a checkpoint, including solution code, score, `solution_id`, and `parent_id`.
- `database/checkpoints/.../metadata.json` stores population state and selection metadata.
- `database/checkpoints/.../solutions/` stores candidates admitted into the evolutionary database.
- `iteration/{K}/planner/` stores planner prompts and selected plans.
- `iteration/{K}/executor/` stores generated candidate code, histories, and best candidate for that iteration.
- `iteration/{K}/summarizer/` stores the feedback used by later planner steps.
- `evaluator/eval_<hash>/` stores compiled candidate code and evaluation results.

The winning checkpoints are:

Task	Winning checkpoint
GDN decode	<code>Full-agent-trace_gdn_decode/database/checkpoints/checkpoint-checkpoint-iter-10-10/best_solution.json</code>
GDN prefill	<code>Full-agent-trace_gdn_prefill/database/checkpoints/checkpoint-checkpoint-iter-10-10/best_solution.json</code>
DSA sparse attention	<code>Full-agent-trace_sparse_attn/database/checkpoints/checkpoint-checkpoint-iter-20-20/best_solution.json</code>
DSA top-k indexer	<code>Full-agent-trace_topk_indexer/database/checkpoints/checkpoint-checkpoint-iter-14-13/best_solution.json</code>
MoE	<code>Full_agent_trace/database/checkpoints/checkpoint-checkpoint-iter-40-37/best_solution.json</code>

D SELECTED NCU PROFILING EVIDENCE

NCU records were compressed into decision evidence rather than used as a promotion metric. Each usable profile was summarized by target region, launch scale, waves per SM, achieved occupancy, memory/compute throughput, and candidate headroom. The evaluator still made the final decision with compile success, correctness, and measured kernel latency.

For GDN decode, early profiles showed launch-limited kernels with low waves per SM and low achieved occupancy. This motivated additional block-level parallelism, but later latency results showed that occupancy alone was not the objective; the search shifted to fixed-shape specialization, register reductions, and deterministic dispatch.

For DSA sparse attention, early profiles exposed short sparse-attention kernels where grid size, waves per SM, and merge overhead limited the split-K route. The planner therefore kept split-K as a robust fallback while asking for fused online-softmax paths and reduced launch/merge overhead.

For MoE FP8 routing, profiles captured routing, elementwise, and grouped-GEMM regions together. The evidence was useful for attribution: it separated routing overhead from GEMM work and guided prompt diagnosis, while later promotion relied on exactness checks and latency deltas.